

Evoluindo com TypeScript

Resumo dos conceitos e guia prático de aprendizado organizado por arquivos e tópicos.

1. Tipos Básicos

1.1 Number `type_number.ts`

O tipo `number` representa números inteiros e decimais (ponto flutuante), além de aceitar valores representados em formato hexadecimal e binário.

```
let valor1: number = 10;
valor1 = 20.9;

let valor2: number = 15;
console.log(valor1 + valor2);
```

1.2 Boolean `type_boolean.ts`

O tipo `boolean` aceita apenas `true` ou `false`. É possível usar a função global `Boolean()` para conversão de valores. Valores como `0`, string vazia `""`, `null` e `undefined` são avaliados como `false`.

1.3 Null e Undefined `type_null.ts`

Representam ausência de valor. Permite-se declarar unions para aceitar múltiplos tipos alternativos:

```
let nome: string | null;
let nomeUser: undefined | number;
```

Diferença principal: `null` representa uma ausência intencional de valor, enquanto `undefined` indica que a

2. Objetos e Coleções

2.1 Object `type_object.ts`

O tipo `object` permite armazenar valores estruturados não primitivos. Para descrever com exatidão as propriedades e seus tipos, recomenda-se o uso de interfaces ou *type aliases*.

2.2 Arrays `type_array.ts`

Arrays podem ser tipados de duas formas sintáticas equivalentes: `Array<T>` ou `T[]`.

```
let filmes: (string | number)[] = ["Filme 1", "Filme 2"];
filmes.push(15); // Aceito devido ao union type
```

2.3 Tuplas `type_tupla.ts`

Uma tupla é um array com número fixo de elementos cujos tipos são conhecidos e fixados em posições específicas.

```
let aluno: [string, number];
aluno = ["Sujeito Programador", 123];
```

Nota de atenção: O método `push()` pode anexar novos itens à tupla sem violar a verificação inicial do compilador, exigindo cuidado no manuseio.

3. Combinação de Tipos e Valores Nomeados

3.1 Union Type `union_type.ts`

Permite declarar que uma variável pode conter mais de um tipo válido. O operador `typeof` pode ser utilizado para checar o tipo em tempo de execução.

```
let userId: number | string;
userId = 123;
```

3.2 Enum `type_enum.ts`

Enums permitem definir um conjunto de constantes nomeadas. Quando os valores não são definidos explicitamente, o TypeScript atribui valores numéricos sequenciais iniciando em `0`.

```
enum DesignColors {
  white = "#FFFFFF",
  black = "#000000",
}
```

4. Tipos Flexíveis e Verificação

4.1 Unknown vs Any `type_unknown.ts` / `type_any.ts`

`unknown` é a alternativa segura para `any`. Um valor de tipo `unknown` não pode ser atribuído a outras variáveis nem ter métodos chamados sem antes passar por uma verificação de tipo ou asserção.

Em contrapartida, `any` desativa a verificação do compilador, podendo causar erros em tempo de execução.

4.2 Type Assertions `type_assertions.ts`

Permite orientar o compilador sobre o tipo exato de uma variável quando você possui mais informações do que o sistema de inferência do TypeScript.

```
let statusAtual: unknown = 1;
let mudaStatus: number = statusAtual as number;
// Sintaxe alternativa: <number>statusAtual
```

5. Funções

5.1 Tipagem de Parâmetros e Retorno `funcoes.ts`

```
function soma(valor1: number, valor2: number): string {
  const soma = valor1 + valor2;
  if (soma > 40) return "SOMA MAIOR QUE 40";
  return "MENOR QUE 20";
}
```

5.2 Arrow Functions `arrowfunctions.ts`

```
const retornoAPI = (url: string): void => {
  console.log("URL DA API: ", url);
};
```

5.3 Parâmetros Padrão e Opcionais `valor_default.ts`

```
function cadastro(
  email: string,
  senha: string,
  nome = "Aluno",
  idade?: number
): void {
  const data = { email, senha, nome, idade };
  console.log(data);
}
```

5.4 Rest Parameters `rest.ts`

```
function totalVendas(...vendas: number[]): void {
  console.log(`Você fez ${vendas.length} vendas hoje!`);
}
```

6. Sequência Recomendada de Aprendizado

Siga a ordem dos arquivos abaixo para uma evolução gradual do básico ao avançado:

#	Arquivo	Conceito Chave
1	<code>type_number.ts</code>	Tipagem numérica (inteiros, decimais, hex, bin)
2	<code>type_boolean.ts</code>	Valores booleanos e conversão com Boolean()
3	<code>type_null.ts</code>	Ausência intencional e não definida de valores
4	<code>type_object.ts</code>	Estruturas de objetos
5	<code>type_array.ts</code>	Coleções e listas com múltiplos tipos
6	<code>type_tupla.ts</code>	Arrays com posições e tipos fixos
7	<code>union_type.ts</code>	Combinação e flexibilização de tipos
8	<code>type_enum.ts</code>	Conjuntos de valores nomeados
9	<code>type_unknown.ts</code>	Tipagem desconhecida e segura
10	<code>type_any.ts</code>	Desativação de checagem (usar com cautela)
11	<code>type_assertions.ts</code>	Casting e direcionamento do compilador
12	<code>funcoes.ts</code>	Tipagem de parâmetros e retornos
13	<code>arrowfunctions.ts</code>	Sintaxe de arrow functions tipadas
14	<code>valor_default.ts</code>	Parâmetros opcionais e valores padrão

#	Arquivo	Conceito Chave
15	<code>rest.ts</code>	Parâmetros variáveis com operador rest (...)